Dyna, an Integrated Architecture for Learning, Planning, and Reacting

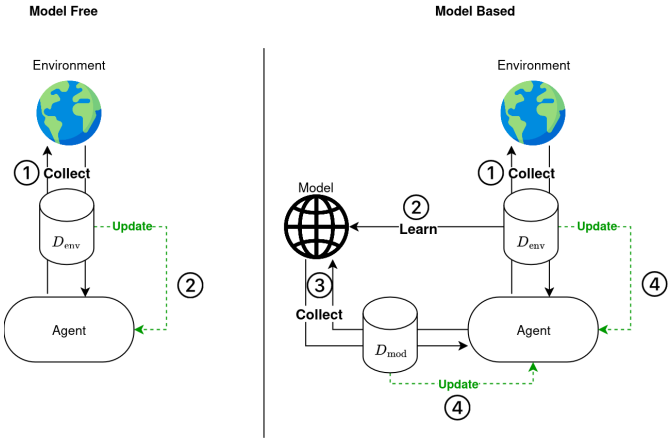
Lorenzo Bonanni

University of Neuchatel

March 3, 2026

Project overview

The project study the problem of **Model Based Reinforcement Learning**



Takeaways

- 👨‍🏫 **Theoretical Knowledge**: ⭐⭐
- 🖥️ **Practical Setup**: ⭐⭐⭐

By working on this project, you will have the opportunity to:

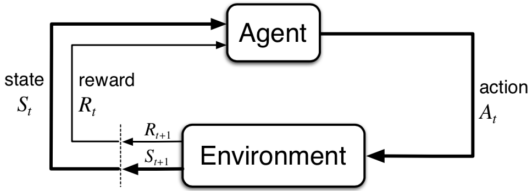
1. Understand how **planning and learning** can be unified within the Dyna architecture.
2. Analyze the **sample efficiency** gains of model-based over model-free RL.
3. Identify the **limits of Dyna-Q** when the model is imperfect or the world changes.
4. Build **Deep Dyna-Q** to scale model-based RL to continuous domains.

Applications of Model Based Reinforcement Learning

The concept of model based reinforcement learning has can applied in various domains:

- **Robotics**
- **Energy Management**
- **Industrial Control**
- **Autonomous Vehicles**

Markov Decision Process

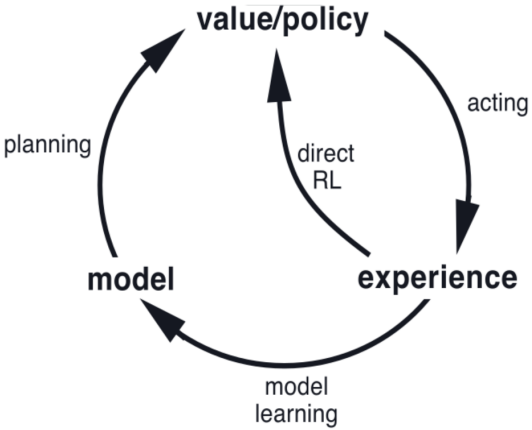


MDP = $\langle \mathcal{S}, \mathcal{A}, \mathcal{R}, \mathcal{P}, \gamma \rangle$

- $\mathcal{S} \rightarrow$ set of possible states
- $\mathcal{A} \rightarrow$ set of possible actions
- $\mathcal{R} \rightarrow$ reward function
- $\mathcal{P} \rightarrow$ transition function
- $\gamma \rightarrow$ discount factor

$$\pi^* = \arg\max_{\pi} \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k r_k \right]$$

Dyna Q


value/policy
acting
planning
direct
RL
model
experience
model
learning

Study Material

- Dyna, an integrated architecture for learning, planning, and reacting
- Reinforcement Learning: An Introduction 2nd Edition, Chapters 8.2 & 8.3

Reproduce Paper

- Implement the Algorithms:
  - Q-Learning (Direct RL)
  - Dyna-Q
  - Dyna-Q+
- Implement the Environments:
  - Dyna Maze
  - Blocking Maze
- Reproduce the Results as shown in the Book:
  - test all the algorithms in all the environments

Deep Dyna-Q Extension

1. Implement Deep Q-Learning
2. Substitute the Tabular Model with a Neural Network Based (assume the reward function is known and not learned, i.e use a fixed function)
3. Compare the Standard Dyna-Q (using discretized states) and the Deep Dyna-Q on the Mountain Car Environment [1]

[1] `https://gymnasium.farama.org/environments/classic_control/mountain_car/`

Useful Resources

- **Deep Q-Learning**
  - Hugging Face
  - Pytorch
- **Neural Networks**
  - IBM-Neural Networks
  - IBM-Backpropagation
  - Pytorch-Neural Networks
  - Mean Square Error

Thank You for Your Attention!

References I